

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 3 : Session – 3 : SLO – 1 | Linked-List Stack — Advantages, Disadvantages, Applications

■ I. Advantages of Pointer-Based Stacks

- Stack implementation using pointers allows DYNAMIC memory allocation.
- In pointer-based stacks, there is no need to define the MAXIMUM stack size.
- Stack implemented using pointers can easily grow and SHRINK at runtime.
- There is no WASTAGE of memory since space is allocated as needed.
- Linked list-based stacks can handle a large number of elements until the system runs out of MEMORY.
- Pointer-based stack avoids STACK overflow, unlike array-based stacks.
- In pointer-based implementation, memory is released using FREE() when elements are popped.
- Stacks using pointers are more FLEXIBLE for programs requiring flexible memory use.
- No need to RESIZE (reallocate) the stack when it becomes full, unlike arrays.
- Each node in a linked list-based stack contains both data and a POINTER to the next node.

■ II. Disadvantages of Pointer-Based Stacks

- Stack implementation using pointers requires extra memory for storing POINTERS.
- Each node in a linked list-based stack uses more MEMORY than array-based stacks.
- Accessing elements in a pointer-based stack is SLOWER than in an array.
- Stack operations in linked lists involve dynamic memory allocation, which can be COSTLY (slow).
- Improper memory management in pointer-based stacks can lead to MEMORY leaks.
- Debugging a pointer-based stack is more DIFFICULT than an array-based stack.
- Pointer-based stack implementation depends on DYNAMIC allocation at runtime.
- Excessive use of pointers increases the risk of RUNTIME (pointer) errors.
- It is harder to determine the total number of elements without TRAVERSING through the stack.
- Stack operations in pointer-based implementation are not as FAST (efficient) as array-based stacks for small data sets.

■ III. Applications

- Stack using pointers is useful in handling VARIABLE (dynamic) size data efficiently.
- Function call management in recursion uses a CALL stack.
- Pointer-based stacks are suitable for evaluating and converting EXPRESSIONS (infix/postfix).
- Stacks help in implementing UNDO and redo features in applications.
- Pointer-based stacks are used in SYNTAX parsing for compilers.
- Backtracking algorithms like maze solving or N-Queens use STACKS.
- Stack using pointers is helpful in managing ACTIVATION (stack) frames in programming languages.
- Depth-first search (DFS) in graphs can be implemented using a STACK.
- Stacks are used in checking BALANCING of parentheses in expressions.
- Linked list-based stacks are useful in systems with limited or unpredictable MEMORY availability.